RAPPORT :

ARBRE BINOMIAL

Encadré par : Mme
Matière : MATHEMATIQUES POUR LA FINANCE

Réalisé par :
— Charly-Romy TANGA

CY TECH

Ingénieur 3ème année
INGÉNIEUR MATHÉMATIQUES OPTION FINANCE ET TECHNOLOGIE

Année Universitaire : 2025-2026

Table des matières

Introduction **2**

Pseudo-code **3**

Sorties **4**

Annexes **6**

Introduction

Contexte

Espace de probabilité sans risque neutre

L'évaluation de la prime de l'option V_0 repose sur l'idée d'un **espace de probabilité neutre au risque**. On définit la probabilité neutre q par :

$$q = \frac{(1+r) - d}{u - d}$$

Cette probabilité permet de calculer l'espérance *actualisée* des flux futurs de l'option, dans un monde sans arbitrage.

avec :

— u : facteur de hausse
— d : facteur de baisse
— r : taux sans risque par période
— $q = \frac{(1+r)-d}{u-d}$: probabilité neutre au risque

Lien avec la couverture et le théorème fondamental

La valeur initiale de l'option V_0 peut être déterminée en construisant un portefeuille **couvert**, combinant l'actif sous-jacent et une position sans risque. Cette méthode garantit que le prix de l'option respecte le **théorème fondamental de la valorisation** et ne permet pas de profiter d'une **opportunité d'arbitrage**.

Une **option** dans le modèle binomial est un produit dérivé dont la valeur dépend du prix de l'actif sous-jacent. On considère la forme dynamique suivante pour une option européenne :

$$V_{n,i} = \frac{1}{1+r}\,\mathbb{E}^{\mathbb{Q}}\big[V_{n+1} \mid \mathcal{F}_n\big] = \frac{1}{1+r}\Big[q\,V_{n+1,i+1} + (1-q)\,V_{n+1,i}\Big]$$

où :

— $\mathbb{E}^{\mathbb{Q}}[\cdot \mid \mathcal{F}_n]$: espérance conditionnelle sous la probabilité neutre au risque q
— $V_{n+1,i+1}, V_{n+1,i}$: valeurs futures de l'option selon l'état up/down
— $1/(1+r)$: actualisation à la période précédente
— $V_{n,i}$: valeur de l'option à l'étape n, état i

Le **delta** au nœud (i,j) se calcule par :

$$\Delta_{i,j} = \frac{V_{i+1,j+1} - V_{i+1,j}}{S_{i+1,j+1} - S_{i+1,j}}$$

Pour illustrer, on considère $T = 2$ périodes et $N = 2$ étapes. L'objectif est de construire l'arbre binomial et de calculer V_0, V_2, ainsi que Δ à chaque nœud.

Pseudo-code

```
# Param tres : S0, K, r, u, d, T, N, option_type
# Initialisation
dt = T / N
R = exp(r*dt)
p = (R - d)/(u - d)
# Arbre des prix du sous-jacent
S = zeros((N+1, N+1))
for i in range(N+1):
    for j in range(i+1):
        S[j,i] = S0 * u**j * d**(i-j)
# Arbre des valeurs de l'option    maturit
V = zeros_like(S)
for j in range(N+1):
    if option_type == 'call':
        V[j,N] = max(S[j,N] - K, 0)
    else:
        V[j,N] = max(K - S[j,N], 0)
# R currence backward pour V
for i in range(N-1, -1, -1):
    for j in range(i+1):
        V[j,i] = (p*V[j+1,i+1] + (1-p)*V[j,i+1])/R
# Calcul du delta au n ud (i,j)
Delta = NaN * ones_like(S)
for i in range(N):
    for j in range(i+1):
        Delta[j,i] = (V[j+1,i+1] - V[j,i+1]) / (S[j+1,i+1] - S[j,i+1])
# R sultat : S, V, Delta, V0, Delta0, V2
```

Sorties

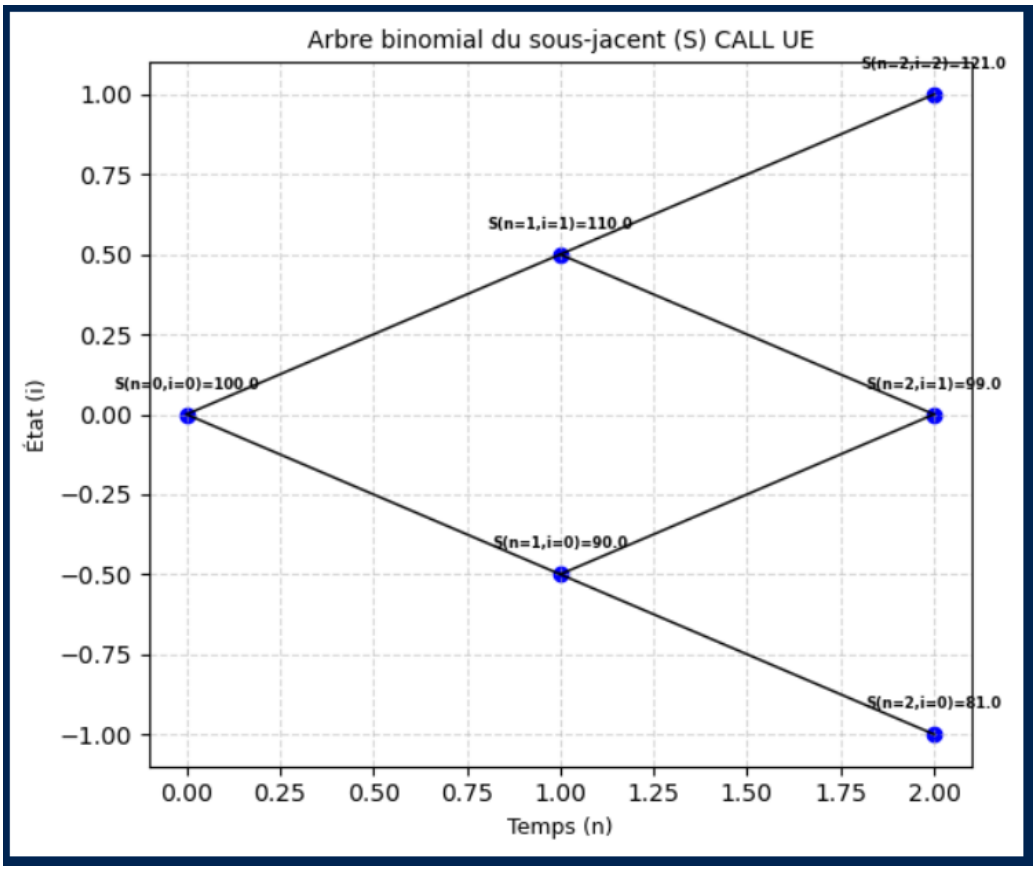

FIGURE 1 – Arbre complet : prix du sous-jacent (S), valeurs de l'option (V) et Delta à chaque nœud pour le CALL

S_0	S_1	S_2	V_0	V_1	V_2	Delta_0	Delta_1	Delta_2
100	90	81	10.87031	0	0	0.755441	0	
0	110	99	0	15.10882	0		0.954545	
0	0	121	0	0	21			

FIGURE 2 – Tableau complet : prix du sous-jacent (S), valeurs de l'option (V) et Delta à chaque nœud pour le CALL

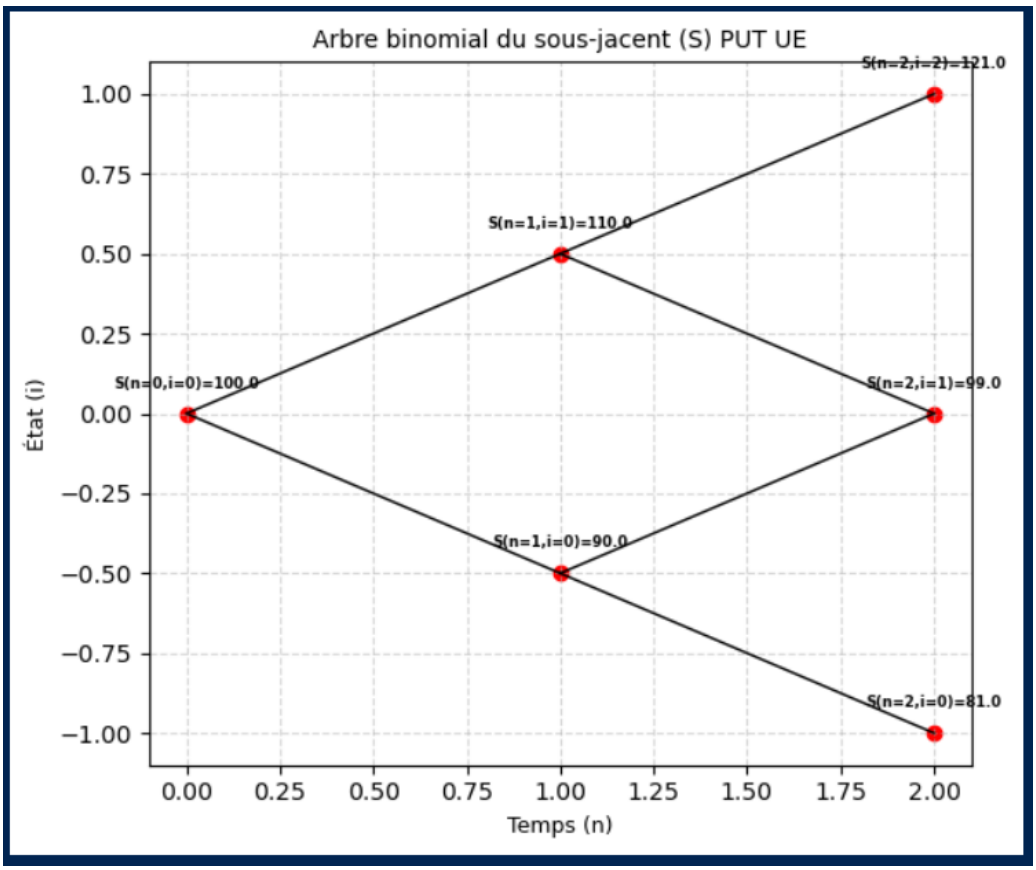

FIGURE 3 – Arbre complet : prix du sous-jacent (S), valeurs de l'option (V) et Delta à chaque nœud pour le PUT

S_0	S_1	S_2	V_0	V_1	V_2	Delta_0	Delta_1	Delta_2
100	90	81	1.354048	5.122942	19	-0.24456	-1	
0	110	99	0	0.231762	1		-0.04545	
0	0	121	0	0	0			

FIGURE 4 – Tableau complet : prix du sous-jacent (S), valeurs de l'option (V) et Delta à chaque nœud pour le PUT

Annexes

```python
def binomial_option_tree(S0, K, r, u, d, T, N, option_type='call'):
    dt = T / N
    R = np.exp(r * dt)        # facteur de croissance sans risque par étape
    p = (R - d) / (u - d)     # proba neutre au risque
    # --- Arbre des prix du sous-jacent ---
    S = np.zeros((N+1, N+1))
    for i in range(N+1):
        for j in range(i+1):
            S[j, i] = S0 * (u**j) * (d**(i-j))
    # --- Valeur de l'option à maturité ---
    V = np.zeros_like(S)
    if option_type == 'call':
        V[:N+1, N] = np.maximum(S[:N+1, N] - K, 0)
    else:
        V[:N+1, N] = np.maximum(K - S[:N+1, N], 0)
    # --- induction ---
    for i in range(N-1, -1, -1):
        for j in range(i+1):
            V[j, i] = (p * V[j+1, i+1] + (1-p) * V[j, i+1]) / R
    # --- Delta à t=0 ---
    delta0 = (V[1,1] - V[0,1]) / (S[1,1] - S[0,1])
    return S, V, V[0,0], delta0
```

[16] ✓ 0.0s

FIGURE 5 – Code Python complet pour l'option